



4

7

840 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Session & Cookies and Software Architecture

Programming Languages



د. روان قرعوني

محتوى مجاني غير مخصص للبيع التجاري

RB Informatcs

10/11/2024

لغات البرمجة

Session & Cookies

إن بروتوكول Http هو Stateless (لا يقوم بتخزين حالة المستخدم) مما يعني أنه لا يحتفظ بمعلومات عن زيارتك السابقة أي عندما تدخل إلى الموقع مرة أخرى يعتبر دخلك كأنه الدخول الأول وليتم تخزين حالة المستخدم لدينا أحد الطريقتين: إما Session أو Cookies.

لنوضح ذلك:

- عند تصفح موقع ما وكانت لغته الافتراضية هي اللغة الإنكليزية وقمنا بتحويلها إلى اللغة العربية مثلاً، فعندما نزر هذا الموقع مرة أخرى أن اللغة لم تتغير عن آخر تحويل قمنا بإجرائه. أي أنه يقوم بالاحتفاظ بآخر التعديلات التي قام بها المستخدم.
 - عند الدخول إلى YouTube نلاحظ أن الـ Videos المقترحة مشابهة لما تمت مشاهدته سابقاً، أي أن الـ YouTube قام بتخزين معلومات عن الـ user حتى دون القيام بـ Login.
- أي يتم في الـ Session والـ Cookies تخزين حالة المستخدم أو معلومات عنه لسهولة التصفح حيث تم اقتراح محتوى مشابه لما تم البحث عنه مسبقاً.

الجدول التالي يوضح الفرق بين الـ Session و Cookies:

Cookies	Session	
Local يتم تخزينها على الحاسب الخاص ضمن مسار معين	يتم تخزينها على الـ Server	مكان التخزين
حجم التخزين قليل أي لا يمكن تخزين 1Gb على الحاسب الشخصي أي يتم تخزين فقط المعلومات البسيطة مثل الـ Username	حجم التخزين كبير	الحجم
غير آمنة لأن الـ Data تخزن على الحاسوب	مستوى الأمان أكبر من الـ Cookies لأنها مخزنة على الـ Server	الأمان
لها باراميتري يدعى Life Time ويقصد به المدة التي ستعيشها الـ Cookies وهي مدة يحددها المبرمج فيمكن أن تكون يوم أو أسبوع أو مدة غير محددة	تنتهي (تغلق) عند عمل Logout أو إغلاق المتصفح	المدة (الديمومة)

أمثلة على استخدام الـ Cookies:

- عند الدخول إلى موقع ما يطلب تسجيل الدخول نقوم بتعبئة حقول الـ Username والـ Password فعند فتح هذا الموقع مرة ثانية ستجد اقتراحات للحقول التي قمت بتعبئتها في وقت سابق أي يتم تعبئة حقول الـ Username والـ Password بشكل تلقائي في حال وافق المستخدم على ذلك.
 - عند القيام بتغيير طريقة عرض موقع ما بين الـ Dark mode والـ Light mode يتم الاحتفاظ بآخر طريقة قمت بها بعرض هذا الموقع فعند فتح الموقع مرة أخرى نجد طريقة العرض هي آخر طريقة قمت بتفعيلها.
 - عند الدخول إلى تطبيق معين وتخييرنا بين اللغة الإنكليزية والعربية، إذا قمنا باختيار اللغة الإنكليزية مثلاً وغادرنا التطبيق (أغلقناه) ودخلنا إليه مجدداً سنجد أن لغة التطبيق بقيت إنكليزية كما اخترناها.
- جميع الأمثلة السابقة تم استخدام الـ Cookies فيها لان حجمها قليل ولا تتطلب حماية عالية.**

مثال على استخدام الـ Session:

يتم استخدام الـ Session في مواقع وتطبيقات التجارة الالكترونية كالتسوق عبر الانترنت حيث يوجد ما يسمى بالـ Cart والذي يمثل عربة التسوق الخاصة بالـ User حيث يقوم المستخدم بالتجول ضمن المتجر وإضافة العديد من المنتجات إلى الـ Cart الخاصة به،

وعند انتهائه يكون أمام حالتين:

1. شراء المنتجات التي وضعها في الـ Cart والخروج من الموقع، وهذا يؤدي إلى حذف محتوى الـ Session.
2. عدم شراء ما تم وضعه بالـ Cart وقام بالخروج من الموقع أو التطبيق وهنا تتم إعادة المنتجات وتفرغ الـ Cart وأيضاً تفرغ محتوى الـ Session.

وهنا تم استخدام الـ Session لأن حجم الـ Cart غير محدد.

- يتم استخدام الـ Session عندما تحتاج إلى تخزين كبير أو عندما تكون المعلومات المراد تخزينها سرية لأن الـ Session أكثر أمناً وحماية.

ملاحظة: يوجد نوع ثالث من التخزين وهو التخزين ضمن الـ Database.

ذكرنا سابقاً أن مدة حياة الـ Cookies هي مدة يحددها المبرمج وتموت الـ Cookies بانتهاء هذه المدة. والـ Session تنتهي عند مغادرة الموقع أو عند القيام بـ Logout.

أما التخزين ضمن الـ Database يتطلب إجراء Login (إنشاء حساب) على الموقع أو التطبيق وعندها يتم التعرف على الـ User في أي وقت ومن أي جهاز يستخدمه ويجرب من خلاله Login على حسابه. فمثلاً كما ذكرنا سابقاً أنه عند الدخول إلى الـ YouTube دون القيام بـ login سيظهر محتوى مشابه لآخر ما تم البحث عنه على الجهاز ويختلف المحتوى المقترح باختلاف الأجهزة واختلاف عمليات البحث بين الأجهزة فهو لا يتعرف على المستخدم وما يستهويه من محتوى.

أما عند القيام بـ login أصبح الوضع مختلف تماماً لأنه تعرف على الـ user وقام بتسجيل بياناته ضمن الـ Database من (اسم، كلمة السر، المحتوى المفضل، آخر ما تم مشاهدته..)، وعندها عندما يقوم المستخدم بالدخول لحسابه من أي جهاز آخر سيتم التعرف عليه وسيظهر المحتوى المفضل لديه ولو قام بتغيير الجهاز مع إجراء عملية login.

Software Architecture Patterns

- سابقاً عند كتابة كود لتطبيق ما ويب أو موبايل من قبل فريق عمل برمجي فقد كان كل مبرمج يكتبها بطريقة الخاصة ويتعامل معها بأسلوبه الخاص فعند دمج الأكواد التي تكون التطبيق كاملاً تبدأ المشاكل بالظهور ضمن الكود ولحل مشكلة الضياع وعدم التوافق بين هذه الأكواد، تم تعيين team leader ضمن الشركة الواحدة دوره القيام بإيجاد طريقة عمل Standard بين أعضاء الفريق وهذا سهل العمل بين المبرمجين ضمن الشركة وحل جزءاً من المشكلة لكن ظهرت مشكلة أخرى وهي صعوبة قراءة الكود وآلية عمله في حال تم تسليمه الشركة أخرى أو توظيف مبرمج جديد في الشركة.
- وهنا تم الاتفاق عالمياً على Patterns محددة لتوحيد أنماط كتابة الكود "Software Architecture". فظهر مفهوم الـ "Software Architecture Patterns". وسنذكر 3 أنواع:

MVC (Model View Controller)

MVP (Model View Presenter)

MVVM (Model View View Model)

- لا يوجد أفضلية بين الأنواع السابقة فهي طريقة عمل محددة بين المطورين. سنقوم بشرح الأنماط السابقة عن طريق مثال ونوضح آلية التواصل:

ليكن لدينا شخص يريد شراء مشروب غازي فيمكنه الشراء عن طريق خيارات عديدة ستعرضها:

1. أولاً MVC:

- وهي أن يذهب الى المتجر فيقوم بطلب المشروب الغازي من البائع وهنا إما البائع سيحضر الطلب للزبون ويقدمه له أو سيقوم الزبون بإخبار البائع بطلبه لكن هو من يجلب الطلب بنفسه.



View

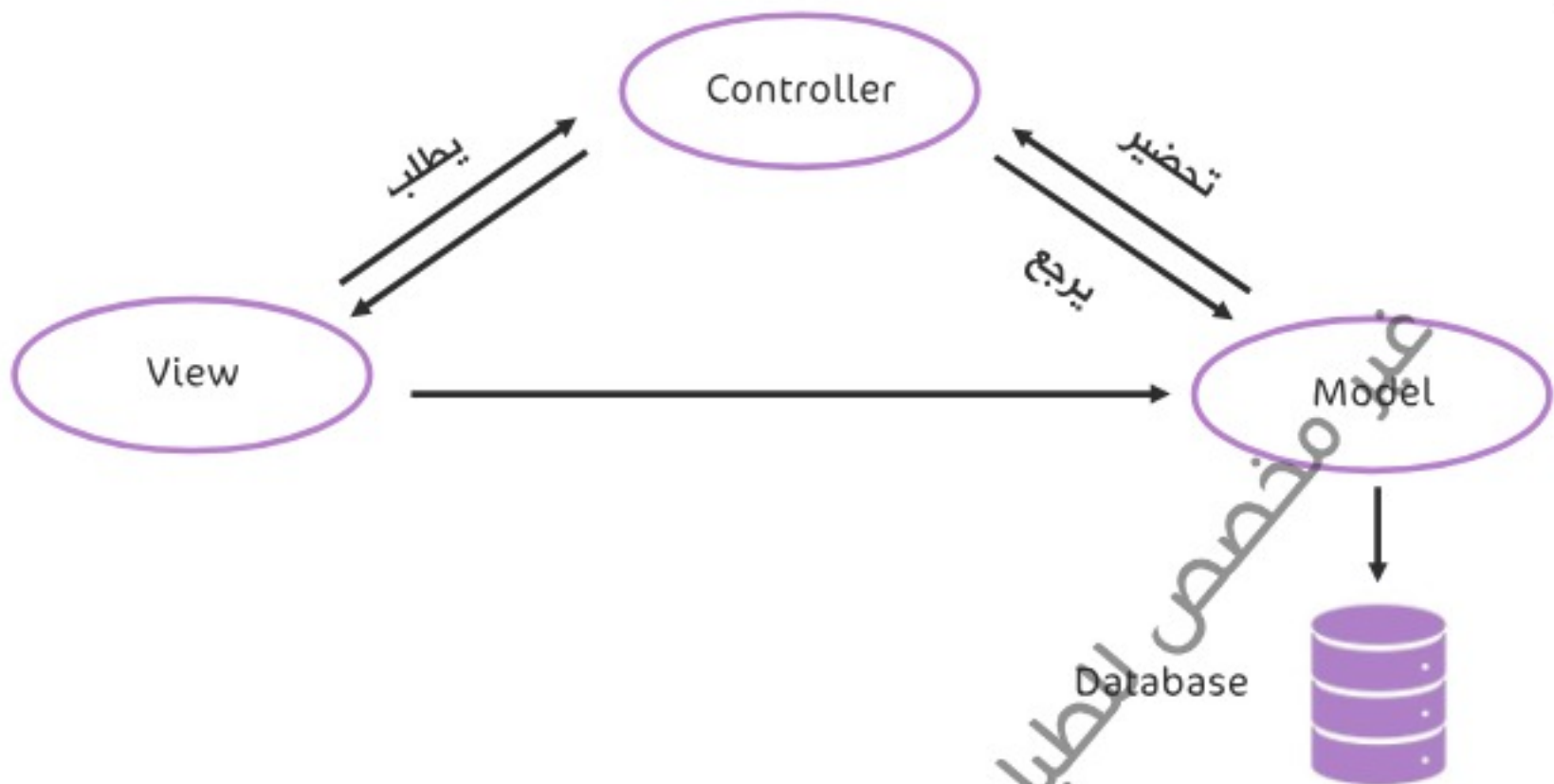


Controller



Model

إن الشخص يمثل الـ view والبائع يمثل الـ controller الذي يقوم بتجهيز الطلب، والطلب (المشروب الغازي) يمثل الـ model.



■ آلية التواصل:

يقوم الـ View بطلب Action معينة من Controller حيث يقوم الـ Controller بطلب Model وتجهيزها عبر Method وعندما ينتهي يقوم الـ Controller بإرجاع الـ Model للـ View أو يستطيع الـ View التواصل عندها مع الـ Model مباشرة

Components of MVC:

Controller

هو الذي يتعامل مع الدخل من المستخدم ويكون صلة الوصل بين الـ View والـ Model حيث يستقبل الدخل من الـ View وينقله للـ Model عن طريق إحدى الـ Methods ثم يعيد النتيجة المناسبة من الـ Model للـ View.

View

هي التي تعرض المعلومات والبيانات في التطبيق للمستخدم وفيها يتم التفاعل بين المستخدم والتطبيق كما يمكن الوصول للـ Model لكن تتم عملياتها عبر الـ Controller ويكون الـ View عبارة عن صفحات الـ Front.

Model

هو المكون الأساسي للتطبيق الذي يحوي كل الـ Data والـ Logic الخاص بالتطبيق ومثل على ذلك تطبيق لعرض تفاصيل سيارات معينة لذلك يتم تخزين البيانات مثل اللون والماركة والسعر ضمن كلاس معين.

- بالعودة إلى المثال السابق فإن أحد الخيارات المتاحة أيضاً هي الذهاب إلى المطعم والجلوس على طاولة وطلب المشروب الغازي من النادل فهو غير مصرح له بإحضار ما يريد بنفسه (وهنا قام النادل بجلب المشروب الغازي خصيصاً له).



View



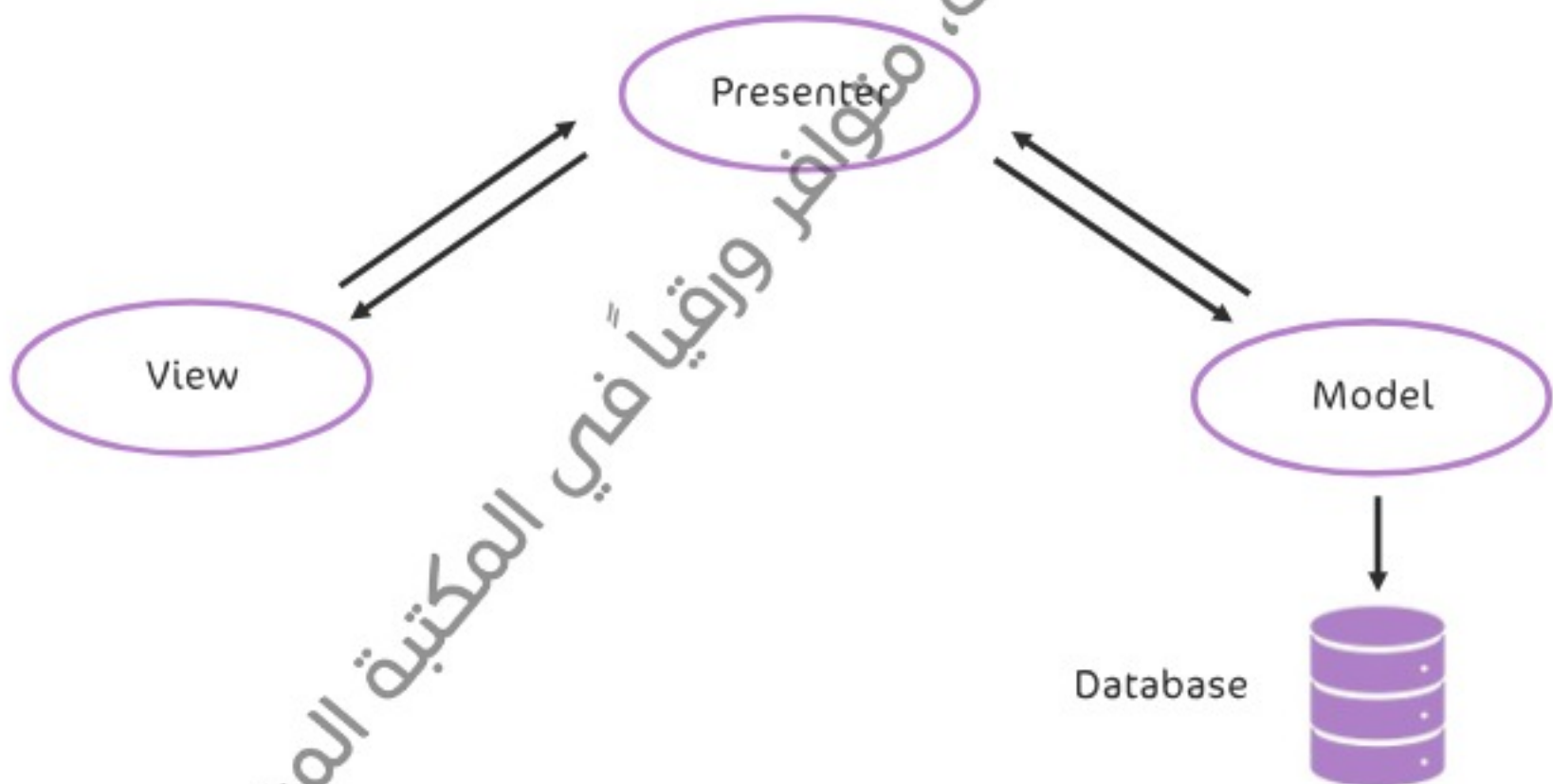
Presenter



Model

آلية التواصل:

يقوم ال View بطلب الامر من ال Presenter ويقوم ال Presenter بالتواصل مع ال Model لإحضار الطلب من ال Database وإيصاله ال Presenter ويقوم ال Presenter بدوره في إيصال الطلب ال View، أي ان ال View لا يستطيع أن يتجاوز ال Presenter ويصل إلى ال Model وال Presenter في هذه الحالة يعرف ال View الذي قام بطلب الأوامر



ملاحظة: في الحياة العملية سنتطرق للاختصار MVP (Minimum Viable Product) أي منتج صغير ذو قيمة

يقصد به إصدار مبكر من المنتج وهو نسخة مصغرة عن المشروع الأساسي يملك الميزات والخصائص التي تخدم العميل مؤقتاً. فمثلاً عندما يطلب العميل برنامج لحساب رواتب الموظفين فعليه تزييد المبرمج بمتطلباته مثل (أيام الدوام _ أيام الغياب _ الساعات الإضافية _ الحسومات _ المكافآت _ التأمينات _ الراتب الكلي ...) ولتنفيذ مشروع بهذه المتطلبات نحتاج إلى وقت طويل لكن العميل يريد نسخة مصغرة تلبي حاجته مبدئياً فيتم إنشاء MVP يخدم بحساب الراتب الكلي وأيام الدوام وأيام الغياب، بينما يصبح المشروع جاهز مع كامل المتطلبات. ونوضح أنها لا علاقة لها بالـ MVP (Model View Presenter)

- بالعودة مرة أخيرة لرؤية الخيارات المتاحة أمام الشخص الذي يريد الحصول على المشروب الغازي فنجد أن بإمكانه الحصول عليه عن طريق آلة العصائر التي تقدم خدمة ذاتية فهي تستقبل من الزبون النقود والطلب وتقوم بإيصال الطلب إلى الدرج الموجود ضمنها لكن هنا لا تستطيع الآلة التعرف على الشخص الذي يستقبل الطلب فيما إذا كان نفسه الذي قام بالطلب أو شخص آخر لا علاقة له.



View



View Model

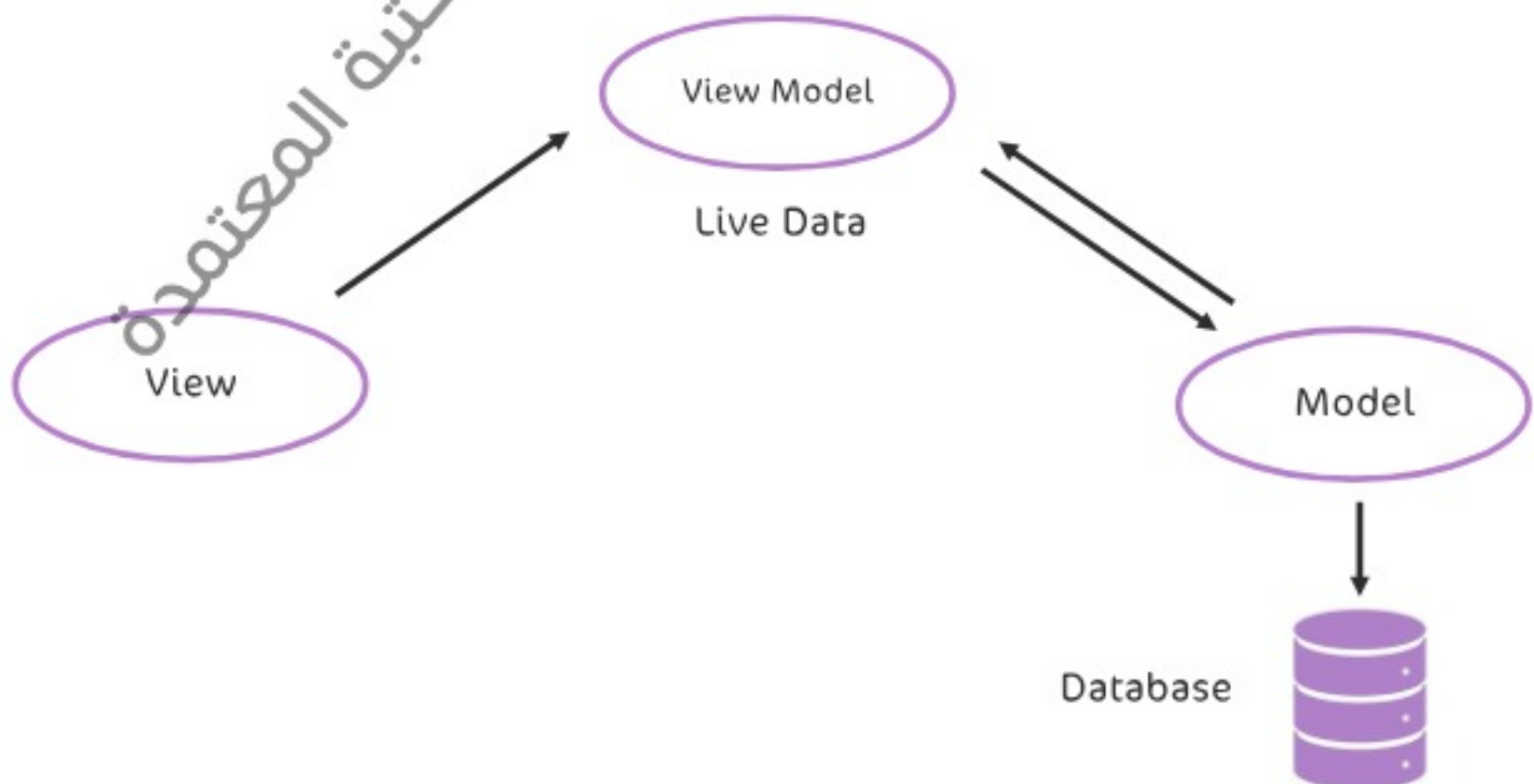


Model

في هذه الحالة يكون الشخص هو الـ View والآلة تمثل الـ View Model والمشروب الغازي هو الـ Model هنا نلاحظ أن الـ View Model أخذ أمر من الـ View (القطع النقدية + خيار المشروب الغازي) ثم قام بإرسال الـ Model حسب طلب الـ View له لكن آلة العصائر لم تقدم المشروب الغازي خصيصاً للشخص (حيث يمكن أن يطلب شخص ما المشروب الغازي ثم يقوم شخص آخر باستلامه)

نتيجة:

في الـ MVVM يقوم الـ View Model بإحضار الـ Object من الـ Database مثلاً ويضعها بما يسمى الـ Live Data حيث أن بعد كل عملية طلب من الـ View توضع الاستجابة ضمن الـ Live Data التي تتيح لأي View الاستفادة مما تحويه. مع الانتباه لأن الـ Live Data تخزن الـ Data ضمنها لكنها لا تقوم بإرجاعها للـ View الذي قام بطلبها وهنا مهمة الـ View هي مراقبة الـ Live Data للحصول على الـ Data التي يريدتها بنفسه.



1. إن مهمة ال Controller مشابهة لمهمة ال Presenter لكن اختلفت التسمية للتفريق بين ال Pattern المستخدم فقط.
2. يمكن تشبيه ال Live Data بالمستودع الذي يخزن جميع ال models التي طُلبت من جميع ال views, فيتم تخزينهم باسم معين وجميع ال views تستطيع الاستفادة منها. وهنا عندما نحتاج model نعلم انه تم طلبها في صفحات سابقة نستطيع استرجاعها والاستفادة منها دون الحاجة لإعادة طلبها.
3. إن الفرق الأساسي بين أنواع ال Patterns الثلاث هي أن كلا ال MVC وال MVP تقدم الطلب للشخص الذي قام بالطلب أما ال MVVM تقدم الطلب دون التعرف على الشخص الذي سيستلم.

You only lose when you stop fighting.

As long as you're fighting

YOU WILL WIN!

